

Cambios Fase 3 - AhorraGo

Fecha: 2026-05-31

Objetivo

Aumentar la precisión del matching de productos y del comparador sin cambiar la arquitectura principal, sin migrar la base de datos y sin modificar el frontend.

Cambios Aplicados

1. Normalización avanzada

Se amplió `app/normalizacion.py` para reconocer equivalencias de formato:

- Volumen:
 - 1L = 1000ml
 - 1.5L = 1500ml
 - 2L = 2000ml
- Peso:
 - 1kg = 1000g
 - 0.5kg = 500g
 - 250g = 0.25kg
- Cantidad:
 - pack 6
 - 6 un
 - 6 unidades
 - x6

Esto reduce falsos negativos cuando distintos supermercados publican el mismo formato con unidades diferentes.

2. Extractor de atributos estructurados

Se agregó `extraer_atributos(nombre_producto)`, que devuelve señales estructuradas para el matching:

```
{
  "marca": "soprole",
  "volumen": 1000,
  "peso": None,
  "unidad": "ml",
  "cantidad": None,
  "categoria": "leche",
  "sabores": [],
  "tokens": [...]
```

```
}
```

Ejemplo cubierto por tests:

```
extraer_atributos("Leche Soprole Entera 1L")
```

Devuelve marca soprole, volumen 1000, unidad ml y categoría leche.

3. Matching Score 0-100

Se creó `matching_score(producto, candidato)` en `app/matching.py`.

El score combina:

- Marca.
- Volumen o peso normalizado.
- Cantidad o pack.
- Categoría/familia.
- `producto_base`.
- Intersección de palabras clave.
- Similitud textual con RapidFuzz.

`candidato_compatible` ahora mantiene filtros duros para evitar falsos positivos y usa el score como señal final.

4. RapidFuzz

Se agregó `rapidfuzz` a `requirements.txt` y se usa `fuzz.token_set_ratio` para evitar depender de comparaciones exactas de texto.

Esto mejora casos como:

- Coca Cola vs Coca-Cola.
- Reordenamientos de palabras.
- Diferencias menores en nombres publicados por supermercado.

5. Casos problemáticos cubiertos

Se agregaron tests para:

- Leche Soprole 1L vs Leche Soprole 1000ml.
- Coca Cola 1.5L vs Coca Cola 1500ml.
- Arroz Tucapel 1kg vs Arroz Tucapel 1000g.
- Pack 6 Coca Cola vs Coca Cola x6.
- Yogurt de sabores distintos, que no debe coincidir.

6. Fixtures reales

Se creó tests/fixtures/productos_reales.json con ejemplos representativos de Jumbo, Lider y Unimarc.

Incluye pares equivalentes y un caso negativo de yogurt con sabores distintos.

Archivos Modificados o Creados

Modificados:

- app/normalizacion.py
- app/matching.py
- requirements.txt

Creados:

- tests/test_matching_score.py
- tests/test_fixtures_matching.py
- tests/fixtures/productos_reales.json
- CAMBIOS_FASE_3.md

Tests Agregados

- Tests unitarios para extractor de atributos.
- Tests unitarios para score de matching.
- Tests con fixtures reales.
- Tests negativos para evitar coincidencias incorrectas por sabor.

Comandos Ejecutados

```
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m pytest -q
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -m compileall app tests
$env:PATH = "$PWD\venv\Scripts;$env:PATH"; python -c "from app.main import app; print(app.title)"
```

Resultados

- Tests: 19 passed.
- Compilación: OK.
- Import de FastAPI app: OK, salida FastAPI.

Precisión Mejorada

La precisión mejora porque el matching ya no depende solo de texto o formato literal:

- Las unidades se comparan en una representación común.
- Los packs se detectan como cantidad.

- La marca se normaliza antes de comparar.
- RapidFuzz tolera diferencias de orden, guiones y texto adicional.
- Los sabores conocidos actúan como freno para evitar falsos positivos.

Riesgos Pendientes

- El sistema de sabores aún es una lista curada y puede necesitar ampliación con datos reales.
- El umbral de `matching_score >= 68` está calibrado con tests iniciales, pero debe validarse con más productos reales.
- Algunas categorías complejas, como detergentes, papel higiénico y promociones multipack, podrían requerir reglas específicas.
- No se ejecutó el pipeline completo de scrapers para evitar cambios masivos de datos.

Próximos Pasos Recomendados

- 1 Medir precisión con una muestra real etiquetada manualmente.
- 1 Agregar fixtures para detergentes, papel higiénico, aceites y cafés.
- 1 Registrar `matching_score` en diagnósticos internos para revisar falsos positivos.
- 1 Ajustar umbrales por categoría si aparecen patrones distintos.
- 1 Crear un reporte de productos agrupados por `producto_base` para detectar errores de equivalencia.